



Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

Agentic AI for Video Creation

Course Code: C436

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 1.0

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
1.0	29 July 2026	Initial aligned release of the C436 deck, Learner Guide, Lesson Plan, and eight connected labs.	Assigned Course Trainer

TABLE OF CONTENTS

Introduction.....	4
Course Learning Outcomes.....	4
Before You Start — Preparation.....	4
Topic 01 — Getting Started with Agentic AI for Video.....	5
Introduction to Agentic AI for Video Creation.....	5
Popular AI Video, Voice, and Agent Tools.....	7
Writing Effective Prompts for Video.....	8
Designing an End-to-End Video Agent Workflow.....	9
Lab 1 — Design the Video Agent Production Contract and Autonomy Matrix.....	10
Lab 2 — Build and Run the Bounded Video Planning Agent in n8n.....	12
Topic 02 — Scripting and Generating Content with AI.....	15
Researching Trends and Ideas.....	15
Generating Scripts and Storyboards.....	16
Creating Visuals and B-Roll with AI.....	17
Generating Voiceovers and Music.....	18
Lab 3 — Run the Research-to-Script Agent and Approve a Timed Storyboard.....	19
Lab 4 — Build the Visual, Voiceover, and Music Asset Request Pack.....	22
Topic 03 — Editing and Assembling Videos with AI.....	26
Automating Video Editing.....	26
Adding Captions, Effects, and Branding.....	27
Assembling Short-Form Videos.....	28
Reviewing and Refining with AI.....	29
Lab 5 — Assemble and Probe the Captioned Vertical Video.....	30
Lab 6 — Run the Independent Video Review Gate and Repair One Finding.....	33
Topic 04 — Automating and Scaling Video Production.....	37
Building Multi-Step Video Agents.....	37
Publishing and Scheduling Across Platforms.....	38
Analysing Performance.....	39
Scaling Your Video Content Pipeline.....	40
Lab 7 — Orchestrate the Private Release Package with Human Approval.....	41
Lab 8 — Analyse Synthetic Performance and Build the Scaling Control Plan.....	45
Integrated Workflow Wrap-Up.....	49
Next Steps.....	49
Glossary.....	49

Introduction

This Learner Guide accompanies Agentic AI for Video Creation (C436). It is a self-contained study text and practical reference for designing, generating, assembling, reviewing, and scaling a controlled short-form video workflow.

The four topics and eight connected labs follow one synthetic Harbour Bean campaign. Work in order and retain each checkpoint: later labs consume the production contracts, manifests, decisions, and evidence created earlier.

Course Learning Outcomes

- LO1: Explain agentic AI, select suitable video-production tools, write bounded prompts, and design a controlled end-to-end workflow.
- LO2: Build AI-assisted research, scripting, storyboard, visual, voiceover, and music hand-offs from an approved creative brief.
- LO3: Assemble and refine a short-form video through automated editing, captioning, branding, and evidence-based quality checks.
- LO4: Orchestrate multi-step video agents with human approval, prepare safe publishing actions, analyse performance, and plan responsible scale.

Before You Start — Preparation

What you need

- A Windows or macOS laptop with a modern browser and permission to create local folders.
- Access to an approved AI assistant such as ChatGPT or Claude; do not paste confidential data into an unapproved service.
- An n8n Cloud workspace or trainer-provided n8n instance for importing the supplied workflow templates.
- FFmpeg and FFprobe on PATH. On Windows, install with 'winget install --id Gyan.FFmpeg -e'; on macOS, use 'brew install ffmpeg'. Reopen the terminal and verify both commands.
- Optional approved accounts for a video generator and voice service; supplied placeholder assets keep every lab completable without paid generation.
- The repository's labs/assets folder, which contains the synthetic brief, templates, sample analytics, manifests, and workflow JSON.

Verify your setup

Confirm that the course files are readable, n8n opens, and the media tools return a version. Never place real secret values in a prompt, lab file, screenshot, or public repository.

```
ffmpeg -version
```

```
ffprobe -version
```

In n8n, open Workflows > Create Workflow and confirm Import from File is available.

Conventions used in every lab

- Replace placeholders such as <RUN_ID> or <API_KEY> only in an approved credential store or local environment.
- Use the supplied synthetic Harbour Bean data. Do not add real customer, employee, creator, or account data.
- Keep public publishing disabled. The labs produce private or dry-run release packages for review.
- Save accepted artifacts under the named checkpoint path before starting the next lab.
- If an optional generation service is unavailable, use the supplied placeholder media and continue the full control workflow.
- For platform request design, use the current official YouTube Data API documentation at <https://developers.google.com/youtube/v3/docs/videos> and TikTok Content Posting API documentation at <https://developers.tiktok.com/doc/content-posting-api-get-started>; the labs keep every request non-executing.

Topic 01 — Getting Started with Agentic AI for Video

Course coverage: Day 1 morning | 2 labs.

Agentic AI foundations | Video, voice, and agent tools | Effective prompts | End-to-end workflow design

Key concepts

- Agent loop — A bounded cycle of observe, plan, act with tools, inspect evidence, and stop or escalate.
- Production contract — A structured brief that fixes the audience, goal, source facts, constraints, deliverables, and approval gates.
- Tool boundary — A named capability with explicit inputs, outputs, permissions, cost limits, and failure behavior.
- Human control — People approve high-impact creative, rights, privacy, brand, and publishing decisions.

Introduction to Agentic AI for Video Creation

Agentic AI combines a model with instructions, tools, state, and a control loop so that the system can decide which bounded action to take next. In video production, an agent may inspect a brief, request missing facts, call research or generation tools, record

outputs, check quality, and route an item for human approval. It is different from a one-shot chatbot because the workflow carries state and can take several tool-mediated steps toward a defined completion condition.

Video work contains creative uncertainty as well as operational dependencies. A reliable agent must know what it may decide, what evidence it must retain, and when it must stop. Treating every step as autonomous creates rights, privacy, cost, and brand risk; treating every step as fixed automation misses the value of reasoning. The practical design is bounded autonomy: deterministic rules for known operations, model judgment for well-framed choices, and human review for consequential actions.

How it works

- Observe the approved brief and current production state.
- Plan the next smallest useful action against a completion checklist.
- Call only an allowed tool with structured inputs and a budget.
- Inspect the returned artifact, provenance, and quality evidence.
- Continue, retry within limits, or escalate to a named human owner.

Worked example

- Harbour Bean needs a 30-second vertical video about three ways to reduce bitter office coffee.
- A coordinator agent checks the brief, delegates research and scripting, then prepares scene requests.
- Every generator result is written to an asset register; publishing remains disabled until a person approves the final package.

Decision guide

Use when	Avoid when
The task has several dependent steps and the next action depends on intermediate evidence.	The goal is vague, the source facts are unapproved, or there is no accountable owner.
Inputs, tool permissions, completion rules, and escalation paths can be stated clearly.	The proposed action would publish, spend, or use a person's likeness without explicit review.

Practitioner quality lens

- Failure signal: The system keeps generating options without reaching a defined finish state.
- Repair move: Add a completion checklist, iteration cap, and escalation rule.
- Quality evidence: The run log shows why each tool was called and who approved the release.

Popular AI Video, Voice, and Agent Tools

A production stack usually separates orchestration from specialist tools. The orchestrator holds the workflow state and routes tasks. Language models research and write. Image or video generators create candidate shots. Speech services produce narration. Editors and command-line media tools assemble, caption, mix, and export. Publishing and analytics APIs act only after credentials and permissions are configured.

Product names and features change, but the jobs remain stable. A portable architecture defines each tool by capability, contract, and fallback instead of hiding the project inside one vendor. Structured outputs reduce broken hand-offs: a script agent should return timed beats, a visual tool should return a file plus provenance, and an editor should receive a validated manifest rather than free-form prose.

How it works

- List each production job and the data it consumes or creates.
- Assign one primary tool and one manual or alternate fallback to each job.
- Define structured fields, file names, size limits, and accepted formats.
- Restrict credentials to the smallest permissions the tool needs.
- Record cost, latency, rights terms, and failure behavior before use.

Worked example

- n8n coordinates eight stages without storing secret values in prompts.
- ChatGPT or Claude creates structured research and script outputs from approved source material.
- A chosen video generator supplies clips, ElevenLabs or an equivalent supplies narration, and FFmpeg or an editor assembles the vertical master.

Decision guide

Use when	Avoid when
A specialist capability materially improves a defined stage and produces a portable output.	A tool requires confidential material that is not approved for that service.
The team has an approved account, a fallback, and a clear rights and privacy position.	The workflow cannot export files, preserve provenance, or cap cost and retries.

Practitioner quality lens

- Failure signal: A downstream stage cannot understand the previous tool's free-form response.
- Repair move: Define a JSON or file manifest contract with required fields.
- Quality evidence: Each stage can be swapped without redesigning the complete workflow.

Writing Effective Prompts for Video

An effective agent prompt is an operating contract, not a slogan. This course uses B-R-I-E-F: Background, Role and responsibility, Inputs and evidence, Execution constraints, and Format plus finish condition. The prompt states which facts are authoritative, which tools may be used, what must never happen, and exactly what valid output looks like.

A creative request such as 'make a viral video' leaves the system to invent the audience, claims, style, and success rule. A structured prompt produces comparable alternatives, exposes missing evidence, and makes hand-offs machine-readable. Separate stable system instructions from per-run input, and validate important fields before a later tool acts on them.

How it works

- Set the audience, business purpose, channel, duration, and desired viewer action.
- Provide approved source facts and mark unknown information explicitly.
- Name the agent's role, allowed tools, cost and iteration limits, and prohibited actions.
- Require a schema for scripts, scenes, claims, assets, risks, and open questions.
- Define a finish condition and conditions that require human clarification.

Worked example

- Background: a Singapore cafe campaign for busy office workers.
- Inputs: only the supplied brand brief and audience signals may support claims.
- Execution: return three 25-35 second concepts; do not publish or imitate a living creator.
- Format: valid JSON with hook, timed beats, scene list, evidence, risks, and status.

Decision guide

Use when	Avoid when
Inputs and desired outputs can be bounded and checked.	The instruction hides a policy decision that an accountable person must make.
Several tools or agents need a common production contract.	The output cannot be validated before a costly or external action follows.

Practitioner quality lens

- Failure signal: The output looks plausible but omits required fields or invents a source.
- Repair move: Add a schema, evidence citations, explicit unknown handling, and a validator.
- Quality evidence: A second run can use the same prompt contract and produce structurally valid output.

Designing an End-to-End Video Agent Workflow

A workflow turns the production contract into observable stages: intake, research, concept selection, script and storyboard, asset generation, assembly, quality review, approval, publishing preparation, and learning. Each stage receives a defined input, writes a durable output, and returns a status such as ready, needs-revision, blocked, or approved.

Large all-in-one agents are difficult to debug and may repeat expensive work. Stage boundaries create checkpoints, allow deterministic validation, and let a learner restart from the last accepted artifact. Idempotency keys prevent a retry from duplicating external actions; run identifiers connect every artifact and log entry to the same job.

How it works

- Draw the stages and mark every external system or generated artifact.
- Define one input/output contract and owner for each stage.
- Place validation before expensive generation and approval before publishing.
- Add retry limits, timeouts, idempotency keys, and a dead-letter or rework route.
- Log run ID, prompt version, tool, cost, result, evidence, and approval decision.

Worked example

- Run HB-001 advances only when brief_status=approved and script_status=approved.
- Scene generation retries once for a technical failure but routes a rights concern to human review.
- The publishing node receives a release package only; it cannot see raw research or use an unapproved file.

Decision guide

Use when	Avoid when
The work has repeatable stages and accepted artifacts can be reused.	The process is a one-off creative conversation with no reusable structure.
The system can save state and resume safely after a failure.	A retry might duplicate an external action and no idempotency or approval control exists.

Practitioner quality lens

- Failure signal: A failed final step forces the whole production to regenerate.
- Repair move: Persist accepted stage outputs and resume from a checkpoint.
- Quality evidence: A run can stop, rejoin, and explain its complete state without guesswork.

Lab 1 — Design the Video Agent Production Contract and Autonomy Matrix

Learning outcome: LO1: explain agentic AI and design a controlled video workflow with explicit human decision points.

Goal: Create the authoritative contract that every later C436 lab will consume.

You will turn the synthetic Harbour Bean campaign brief into a structured production contract, classify each task by autonomy level, and mark the evidence, approval, retry, and stop rules for the workflow.

What you'll build

A completed production-contract.json and autonomy-matrix.csv for run HB-001, saved in the connected project checkpoint. (Tools: Text editor, supplied synthetic brief, JSON validator.)

Prerequisites

- Download or clone the C436 repository and open labs/assets.
- Confirm that harbour-bean-brand-brief.md and production-contract-template.json are present.
- Know the local path to the C436 repository; do not use real customer or account data.

Step-by-step

1. Set the repository root and fail fast if the course assets are absent

```
Set-Location '<PATH_TO_C436_REPOSITORY>'
```

```
if (-not (Test-Path -LiteralPath labs/assets/harbour-bean-brand-brief.md -PathType Leaf)) { throw 'Run this lab from the C436 repository root.' }
```

2. Create the connected project folders

```
New-Item -ItemType Directory -Force -Path  
C436-work/HB-001/01-design,C436-work/HB-001/02-create,C436-work/HB-001/03-  
edit,C436-work/HB-001/04-release,C436-work/HB-001/05-learn | Out-Null
```

3. Read the synthetic brief and identify only approved facts

```
Get-Content -LiteralPath labs/assets/harbour-bean-brand-brief.md
```

4. Copy the contract and autonomy templates into stage folder 01-design

```
Copy-Item -LiteralPath labs/assets/production-contract-template.json -Destination  
C436-work/HB-001/01-design/production-contract.json
```

```
Copy-Item -LiteralPath labs/assets/autonomy-matrix-template.csv -Destination C436-  
work/HB-001/01-design/autonomy-matrix.csv
```

5. Open production-contract.json and replace every text placeholder using only the supplied brief

```
notepad C436-work/HB-001/01-design/production-contract.json
```

6. Set the required contract controls

```
Set run_id to HB-001; retain contract_version contract-v1; duration_seconds 30;
aspect_ratio 9:16; publishing_mode dry_run_private;
max_generation_attempts_per_scene 2; cost_budget_sgd 25.0; finish_condition to
release package approved or routed to named owner. Keep numbers unquoted.
```

7. Complete the autonomy matrix for all listed tasks

Use deterministic for schema validation, file naming, media probing, arithmetic, and packaging; model_assisted for research clustering, script alternatives, and review suggestions; human_approved for claims, rights, final edit, and release; prohibited for secret exposure, unreviewed public posting, and unapproved likeness use.

8. Run the fail-closed Lab 1 validator and retain the evidence

```
PowerShell -ExecutionPolicy Bypass -File labs/assets/validate-lab-checkpoint.ps1
-Lab 1 2>&1 | Tee-Object C436-work/HB-001/01-design/lab-01-test-output.txt
```

9. Check that no template placeholder remains

```
Select-String -Path C436-work/HB-001/01-design/production-contract.json,C436-
work/HB-001/01-design/autonomy-matrix.csv -Pattern '<COMPLETE_ME>'
```

10. Write the Lab checkpoint 01 marker

```
Set-Content -LiteralPath C436-work/HB-001/01-design/LAB-CHECKPOINT-01.txt -Value
'Lab 1 passed; stage folder 01-design is ready.'
```

Test it

The validator must print LAB-01 PASS. It enforces JSON types, positive budget, all nine autonomy rows, prohibition of public publishing, and prohibition of secret exposure. The placeholder search must return no matches.

Checkpoint and rejoin point

Lab checkpoint 01 is stored in stage folder C436-work/HB-001/01-design. If you fall behind, copy labs/assets/production-contract-approved.json and labs/assets/autonomy-matrix-approved.csv into this folder, then rerun the tests before continuing.

Troubleshooting

If this happens	Fix
ConvertFrom-Json reports an invalid object.	Open the file, check the line named in the error, remove trailing commas, and ensure every key and text value is in double quotes.
The brief does not contain a value requested by the template.	Write null or add it to open_questions; do not invent a fact.
A task seems both model-assisted and human-approved.	Classify the model's draft as model_assisted and the consequential decision or action as human_approved.

Challenge

Add a budget warning threshold at 80% of the required cost_budget_sgd and prove the rule returns a named owner rather than another generation attempt.

Reflection

Which video-production decision was most tempting to automate fully, and what evidence convinced you to keep a human approval point?

Note: The complete lab and its support-file references are in labs/lab-01-*.md. Use only the supplied synthetic campaign data and approved accounts. Store secrets in managed credentials, keep public publishing disabled, and obtain human approval before any external release action.

Lab 2 — Build and Run the Bounded Video Planning Agent in n8n

Learning outcome: LO1: use structured prompts and tool boundaries to implement a resumable planning workflow.

Goal: Import, inspect, and run a planning workflow that stops on missing evidence and emits a valid production plan.

You will import a supplied n8n workflow, trace its trigger, validation, planning, guardrail, and output stages, then run one ready case and one blocked case. The agent uses deterministic mock planning so everyone can verify the control pattern without a paid API.

What you'll build

An n8n workflow named C436-HB-001-Planning-Agent plus exported ready and blocked execution evidence. (Tools: n8n, supplied workflow JSON, production contract.)

Prerequisites

- Complete Lab 1 or restore Lab checkpoint 01 in stage folder 01-design.
- Access an n8n Cloud workspace or trainer-provided n8n instance.
- Keep live credentials disconnected; this lab uses deterministic Code nodes.

Step-by-step

1. Open n8n and import the supplied workflow

In n8n, select Workflows > Create Workflow. Open the top-right three-dot menu, select Import from File, and choose `labs/assets/video-planning-agent-workflow.json`.

2. Rename and save the imported workflow

Set the workflow name to `C436-HB-001-Planning-Agent` and select Save. Keep the workflow inactive.

3. Inspect the five-node control path

Confirm the main path is `Manual Trigger > Load Exact Production Contract > Validate Contract and Version > Plan or Block > Emit Run State`. Confirm a blocked result calls no external tool.

4. Paste the exact Lab 1 artifact into the input node

Open `Load Exact Production Contract`. Copy the entire contents of `C436-work/HB-001/01-design/production-contract.json` between the `contractText` backticks. Do not retype or use the supplied approved fixture. Run once and verify `contract_version=contract-v1` plus a non-empty `source_fingerprint`.

5. Execute the complete ready workflow

Select Test Workflow. After execution, select Emit Run State and open the JSON output.

6. Verify the ready-state schema

The output must contain `run_id=HB-001`, `status=plan_ready`, `current_stage=planning`, `next_stage=research_and_script`, a non-empty actions array, `iteration=1`, and `publish_allowed=false`.

7. Create a controlled missing-evidence case

Copy the complete Emit Run State output to C436-work/HB-001/01-design/ready-run.json. Then open Load Exact Production Contract, set approved_facts to an empty array inside the pasted JSON, save, and select Test Workflow again.

8. Verify the workflow stops safely

Open Emit Run State. Confirm status=blocked, next_stage=human_clarification, publish_allowed=false, and blockers includes missing_approved_facts. Save the complete output as C436-work/HB-001/01-design/blocked-run.json.

9. Restore the ready case and export the workflow

Restore the exact approved_facts array from the Lab 1 file, rerun from Manual Trigger, and save the complete output as C436-work/HB-001/01-design/ready-restored-run.json. Download the workflow as planning-agent-reviewed.json.

10. Save a short execution evidence note

```
PowerShell -ExecutionPolicy Bypass -File labs/assets/validate-lab-checkpoint.ps1 -Lab 2 2>&1 | Tee-Object C436-work/HB-001/01-design/lab-02-test-output.txt
```

```
Set-Content C436-work/HB-001/01-design/LAB-CHECKPOINT-02.txt 'Lab 2 passed; planning evidence retained in stage folder 01-design.'
```

Test it

The Lab 2 validator must print LAB-02 PASS after reading ready-run.json, blocked-run.json, and ready-restored-run.json. Both ready results must remain non-publishing; the controlled missing-evidence result must fail closed.

Checkpoint and rejoin point

Lab checkpoint 02 adds three execution JSON files, test evidence, and planning-agent-reviewed.json to stage folder C436-work/HB-001/01-design. The template remains available at labs/assets/video-planning-agent-workflow.json.

Troubleshooting

If this happens	Fix
n8n rejects the imported JSON.	Confirm that you selected the workflow file rather than a course data file, then ask the trainer for the current n8n import fallback.
The path after Validate Contract does not reach Emit Run State.	Open the validation output and confirm required fields use the exact supplied names and approved_facts is an array.
The blocked run still shows plan_ready.	Execute from Manual Trigger after saving the edited contract node; running only the final node may reuse pinned data.

Challenge

Add a deterministic budget check that returns blocker generation_budget_missing when cost_budget_sgd is absent.

Reflection

Why is the blocked result a successful agent behavior rather than a workflow failure?

Note: The complete lab and its support-file references are in labs/lab-02-*.md. Use only the supplied synthetic campaign data and approved accounts. Store secrets in managed credentials, keep public publishing disabled, and obtain human approval before any external release action.

Topic 02 — Scripting and Generating Content with AI

Course coverage: Day 1 afternoon | 2 labs.

Trend and idea research | Scripts and storyboards | Visuals and B-roll | Voiceovers and music

Key concepts

- Source register — A dated record of evidence, relevance, permissions, and claims that may be used.
- Timed beat sheet — A sequence linking narration, visual, on-screen text, sound, and duration.
- Asset manifest — The authoritative list of scene files, prompts, versions, rights notes, and status.
- Continuity bible — Stable character, product, lighting, palette, camera, and negative constraints across generations.

Researching Trends and Ideas

Agentic research is a bounded evidence-gathering workflow. It starts with a question and approved source types, retrieves observations, records provenance, separates evidence from inference, and produces candidate ideas that are traceable to audience need.

Trend signals are clues for timing and format, not permission to copy another creator.

A research agent can gather many examples quickly, but search results may be stale, duplicated, promotional, or detached from the target audience. A source register and evidence threshold prevent a script agent from treating popularity as truth. The output should include open questions and rejected ideas, not only a polished recommendation.

How it works

- State the audience problem, research window, market, and allowed sources.

- Collect dated signals and record source, observation, and confidence separately.
- Cluster repeated needs, questions, formats, and language patterns.
- Generate original angles from the brand's own proof and feasible assets.
- Human-review the shortlist for relevance, truth, rights, and production effort.

Worked example

- Audience signals mention bitter office coffee, inconsistent scoops, and limited time.
- The agent groups these into three teachable variables instead of copying a trending cafe video.
- Each candidate idea cites the supplied observation and the brand fact that supports it.

Decision guide

Use when	Avoid when
The question, time window, source types, and evidence threshold are explicit.	The workflow would scrape personal data or bypass platform access controls.
A person can inspect the source register before ideas move to scripting.	Trend volume is being used as proof for a product or health claim.

Practitioner quality lens

- Failure signal: The idea bank contains unsourced claims and near-copies of examples.
- Repair move: Require provenance, originality notes, and a reason for every retained idea.
- Quality evidence: The chosen idea can be traced to audience evidence and approved brand facts.

Generating Scripts and Storyboards

A script translates the selected promise into spoken words and on-screen text; a storyboard translates the same promise into timed visual evidence. The agent should work in beats rather than a single paragraph. Every beat states time, narration, picture, overlay, sound, claim source, and transition so production and review share one model.

Short-form videos fail when words, visuals, and captions compete or when the final edit exceeds the target duration. Time budgeting before generation constrains scope and avoids unnecessary media cost. A storyboard also exposes scenes that cannot be produced safely or consistently before the system requests them.

How it works

- Lock one hook, promise, evidence set, and viewer action.
- Allocate seconds to hook, proof beats, synthesis, and close.

- Write concise narration and a shorter complementary text overlay.
- Specify a feasible shot, motion, composition, and transition for each beat.
- Run claim, duration, continuity, accessibility, and rights checks before approval.

Worked example

- A 30-second script allocates 3 seconds to the hook, 21 seconds to three fixes, and 6 seconds to recap and close.
- The spoken line explains the fix while on-screen text labels only the variable.
- Every claim points back to an approved source row; the storyboard marks one synthetic scene for disclosure.

Decision guide

Use when	Avoid when
The message and source facts are approved and a duration can be fixed.	The script requires unsupported before-and-after proof or an unapproved likeness.
The next stage needs structured scene requests and voice text.	The storyboard depends on complex continuity the chosen generator cannot maintain.

Practitioner quality lens

- Failure signal: Narration, captions, and visuals repeat the same sentence.
- Repair move: Assign a distinct job to each channel: explain, label, or demonstrate.
- Quality evidence: A cold reader can produce the intended cut from the timed beat sheet.

Creating Visuals and B-Roll with AI

Visual generation converts storyboard beats into candidate images or clips. A strong scene request defines subject, action, environment, composition, camera, lighting, palette, duration, and negative constraints. The asset manifest records the prompt, model or tool, date, version, rights note, and accepted use for every file.

One attractive clip does not create a coherent video. Continuity, editability, and factual fit matter more than isolated novelty. Generate short modular shots, preserve safe areas for vertical overlays, and review anatomy, text, logos, product details, and motion before a clip is accepted.

How it works

- Create a continuity bible from the approved brand and storyboard.
- Generate low-cost candidates or still frames before expensive motion.
- Inspect each result against scene purpose, composition, continuity, and rights.
- Record accepted and rejected versions in the asset manifest.
- Use approved stock or a supplied fallback if generation fails.

Worked example

- Scene 02 shows a measured scoop and timer on a clean office pantry counter.
- The prompt fixes the Harbour Bean palette and leaves the lower third clear for captions.
- A clip with a malformed product label is rejected even if its motion is appealing.

Decision guide

Use when	Avoid when
The scene is synthetic, clearly bounded, and does not require a real person's identity.	The scene impersonates a person, fabricates a real event, or uses protected material without permission.
The team can retain provenance and has a fallback for failed generation.	The required text or exact product geometry should be produced deterministically in the editor.

Practitioner quality lens

- Failure signal: Accepted scenes vary in lighting, product form, and screen direction.
- Repair move: Reuse a continuity bible and generate by shot family with reference frames where allowed.
- Quality evidence: Every accepted file has provenance and supports a named storyboard beat.

Generating Voiceovers and Music

Voice generation turns approved narration into timed audio. Music supports pace and emotion without masking speech. The workflow selects a permitted voice, normalises written text for speech, produces an audio file, checks pronunciation and duration, and records the service, settings, consent, and usage terms.

Audio can make a visually strong video inaccessible or untrustworthy. Unauthorised voice cloning, poor pronunciation, excessive loudness, and unclear licensing create avoidable risk. Voice and music are therefore separate reviewed assets with explicit owners and fallback options.

How it works

- Use an approved synthetic or licensed voice; obtain consent for any personal voice clone.
- Rewrite symbols, dates, acronyms, and names for the intended spoken form.
- Generate a test line, review pronunciation and pace, then render the full narration.
- Choose licensed music or a platform-approved library and retain the rights note.
- Mix for intelligible speech, inspect peaks, and preview on speakers and headphones.

Worked example

- The agent converts '3 fixes in 30 sec' to natural spoken wording before synthesis.
- The narrator file is saved with the script version and chosen settings.
- Background music is ducked under speech and the no-music export remains available as a fallback.

Decision guide

Use when	Avoid when
The voice identity and music rights are explicit and suitable for the intended channel.	The workflow clones a voice without documented permission.
A person can listen to and approve the complete audio before assembly.	Licensing, commercial use, or territorial rights cannot be established.

Practitioner quality lens

- Failure signal: The narration sounds natural but mispronounces the brand or exceeds the scene timing.
- Repair move: Add a pronunciation dictionary, rewrite for speech, and regenerate only the affected line.
- Quality evidence: The approved audio matches the timed script and has a recorded rights basis.

Lab 3 — Run the Research-to-Script Agent and Approve a Timed Storyboard

Learning outcome: LO2: create evidence-linked research, script, caption, and storyboard hand-offs from an approved brief.

Goal: Produce one approved 30-second concept whose claims and scenes are traceable to the supplied evidence.

You will use an approved AI assistant with a structured B-R-I-E-F prompt, or the supplied offline result, to cluster synthetic audience signals, propose distinct concepts, and return a timed script and storyboard. You will validate the schema and manually approve only evidence-linked content.

What you'll build

A source-register.csv, research-script-prompt.txt, script-storyboard.json, captions-script-v1.vtt, and storyboard-approval.md in stage folder 02-create. (Tools: Approved AI assistant or offline fallback, text editor, supplied synthetic data.)

Prerequisites

- Complete Labs 1 and 2 or restore Lab checkpoint 02, including ready-restored-run.json, from stage folder 01-design.

- Confirm labs/assets/audience-signals.csv, harbour-bean-brand-brief.md, and research-script-prompt-template.txt are present.
- Use only the supplied synthetic inputs; do not search for or add personal audience data.

Step-by-step

1. Validate and consume the Lab 2 planning hand-off

```
$plan=Get-Content -Raw C436-work/HB-001/01-design/ready-restored-run.json |
ConvertFrom-Json

if($plan.status -ne 'plan_ready' -or $plan.next_stage -ne 'research_and_script'
-or [string]::IsNullOrEmpty($plan.source_fingerprint)){throw 'Lab 2 planning
hand-off is invalid.'}

$plan | Select-Object run_id,contract_version,source_fingerprint,status,next_stage
```

2. Create the topic 2 checkpoint folder

```
New-Item -ItemType Directory -Force -Path C436-work/HB-001/02-create | Out-Null
```

3. Copy the synthetic evidence and prompt template

```
Copy-Item -LiteralPath labs/assets/audience-signals.csv -Destination
C436-work/HB-001/02-create/source-register.csv

Copy-Item -LiteralPath labs/assets/research-script-prompt-template.txt
-Destination C436-work/HB-001/02-create/research-script-prompt.txt
```

4. Inspect the source register before prompting

```
Import-Csv C436-work/HB-001/02-create/source-register.csv | Format-Table
source_id,observation,evidence_type,allowed_use
```

5. Complete the prompt placeholders from the approved production contract

Open research-script-prompt.txt. Set run ID HB-001 and paste the exact source_fingerprint from ready-restored-run.json. Set the audience, 30-second duration, 9:16 aspect ratio, three concepts, and required JSON schema. Paste the approved facts and source rows.

6. Run the prompt in an approved AI assistant

Start a new chat, paste the completed prompt, and do not enable external actions. If unavailable, run: Copy-Item labs/assets/script-storyboard-approved.json C436-work/HB-001/02-create/script-storyboard.json, then continue at validation.

7. Save only the JSON response

Copy the assistant's JSON object into C436-work/HB-001/02-create/script-storyboard.json. Remove text outside the outer braces and ensure the root contains script_version='script-v1' and plan_source_fingerprint copied exactly from ready-restored-run.json.

8. Validate the JSON and inspect the selected concept

```
Get-Content -Raw C436-work/HB-001/02-create/script-storyboard.json | ConvertFrom-
Json | Select-Object
run_id,plan_source_fingerprint,status,selected_concept_id,total_duration_seconds |
Format-List
```

9. Generate a versioned caption hand-off from the exact storyboard

```
PowerShell -ExecutionPolicy Bypass -File labs/assets/captions-from-storyboard.ps1
-StoryboardPath C436-work/HB-001/02-create/script-storyboard.json -OutputPath
C436-work/HB-001/02-create/captions-script-v1.vtt
```

10. Review every claim and mark the human decision

Create storyboard-approval.md with four headings: Approved concept, Evidence checked, Required revisions, Decision. Set Decision to APPROVED_FOR_ASSET_REQUESTS only after each factual statement maps to an allowed source ID and no scene uses a real person's likeness.

11. Record the approved script version

Add script_version=script-v1 and approved_by=<YOUR_NAME> to storyboard-approval.md. Do not write credentials or personal account identifiers.

12. Run the fail-closed Lab 3 validator and retain the evidence

```
PowerShell -ExecutionPolicy Bypass -File labs/assets/validate-lab-checkpoint.ps1
-Lab 3 2>&1 | Tee-Object C436-work/HB-001/02-create/lab-03-test-output.txt
```

```
Set-Content C436-work/HB-001/02-create/LAB-CHECKPOINT-03.txt 'Lab 3 passed; script
and caption hand-offs accepted.'
```

Test it

The validator must print LAB-03 PASS. It enforces run ID, script_ready status, exact Lab 2 fingerprint hand-off, human approval token, three concepts, required beat fields/source IDs, contiguous timing, and captions.

Checkpoint and rejoin point

Lab checkpoint 03 is stored in stage folder C436-work/HB-001/02-create. The complete rejoin block below restores the approved storyboard, binds it to the exact Lab 2 plan fingerprint, regenerates captions, restores the human approval, and runs the same fail-closed validator.

Troubleshooting

If this happens	Fix
The assistant returns Markdown around the JSON.	Copy only the content from the first opening brace to the final closing brace, then rerun ConvertFrom-Json.
The beats total more than 30 seconds.	Ask for a repair that preserves the approved claim order and reduces narration; do not silently speed up the voice.
A claim has no source ID.	Remove the claim or route it to open_questions. Do not approve it for asset generation.

Challenge

Generate a second concept that uses the same evidence but a different angle family, then compare relevance, proof, feasibility, and estimated generation cost before choosing.

Reflection

Which part of the script required human judgment even after the schema and source checks passed?

Note: The complete lab and its support-file references are in labs/lab-03-*.md. Use only the supplied synthetic campaign data and approved accounts. Store secrets in managed credentials, keep public publishing disabled, and obtain human approval before any external release action.

Lab 4 — Build the Visual, Voiceover, and Music Asset Request Pack

Learning outcome: LO2: create structured media-generation requests with continuity, provenance, rights, and fallback controls.

Goal: Transform the approved storyboard into generator-ready requests without making paid or external actions mandatory.

You will import a deterministic asset-request workflow, generate a continuity bible and one request per storyboard beat, prepare narration and music briefs, and review the resulting asset manifest. Optional live generation is kept outside the required path; supplied placeholder media lets everyone continue.

What you'll build

A continuity-bible.json, asset-requests.json, narration.txt, music-brief.md, and asset-manifest.csv with approved placeholders or authorised generated files. (Tools: n8n, optional approved video/voice tools, supplied placeholder media.)

Prerequisites

- Complete Lab 3 or restore its exact script-storyboard.json, captions-script-v1.vtt, and approval note.
- Confirm labs/assets/asset-request-agent-workflow.json and asset-manifest-template.csv are present.
- If using a live service, store its secret in managed credentials and confirm usage rights and budget with the trainer.

Step-by-step

1. Import the asset-request workflow into n8n

In n8n, select Workflows > Create Workflow, open the top-right three-dot menu, choose Import from File, and select labs/assets/asset-request-agent-workflow.json. Rename it C436-HB-001-Asset-Request-Agent and keep it inactive.

2. Inspect the workflow controls

Confirm the path loads the exact storyboard, validates run_id HB-001 and script_version script-v1, creates one bounded request per beat, caps attempts at 2, and makes zero external actions.

3. Paste the exact Lab 3 storyboard into the workflow

Open Load Exact Approved Storyboard. Copy the entire contents of C436-work/HB-001/02-create/script-storyboard.json between the storyboardText backticks. Run from Manual Trigger and verify source_fingerprint is non-empty.

4. Run the workflow and copy the output

Select Test Workflow. Open Emit Asset Pack and copy the complete JSON output to C436-work/HB-001/02-create/asset-requests.json.

5. Extract the continuity bible

```
$pack = Get-Content -Raw C436-work/HB-001/02-create/asset-requests.json |  
ConvertFrom-Json
```

```
$pack.continuity_bible | ConvertTo-Json -Depth 8 | Set-Content  
C436-work/HB-001/02-create/continuity-bible.json
```

6. Create the narration file from approved beat text

```
$story = Get-Content -Raw C436-work/HB-001/02-create/script-storyboard.json |  
ConvertFrom-Json
```

```
($story.beats.narration -join ' ') | Set-Content  
C436-work/HB-001/02-create/narration.txt
```

7. Prepare the music brief

Copy labs/assets/music-brief-template.md C436-work/HB-001/02-create/music-brief.md. Set mood to warm and practical, duration 30 seconds, dialogue priority high, and permitted source to the supplied course-use placeholder or an approved library. Resolve rights_status to approved_for_course_use before acceptance.

8. Create the asset manifest and register every requested scene

Copy labs/assets/asset-manifest-template.csv C436-work/HB-001/02-create/asset-manifest.csv. Add exactly S01-S05, A01 narration, and A02 music. Every row must be accepted with approved_for_course_use rights. For rejoin, copy asset-manifest-approved.csv.

9. Use supplied placeholder media for the required path

```
New-Item -ItemType Directory -Force C436-work/HB-001/02-create/media | Out-Null
```

```
Copy-Item -Recurse -Force labs/assets/placeholder-media/* C436-work/HB-001/02-  
create/media/
```

10. Optionally replace one placeholder through an approved service

Before a live call, confirm the provider, prompt, estimated cost, rights basis, and credential are approved. Generate only one bounded candidate, save it in media, and update its manifest row. Never paste a secret into the prompt or file.

11. Review the complete pack and record the decision

Create asset-pack-approval.md with checks for storyboard coverage, continuity, narration, music rights, provenance, cost, and fallback. Set Decision to APPROVED_FOR_ASSEMBLY, manifest_version=asset-manifest-v1, and approved_by=<YOUR_NAME> only when every required asset is accepted.

12. Run the fail-closed Lab 4 validator and retain the evidence

```
PowerShell -ExecutionPolicy Bypass -File labs/assets/validate-lab-checkpoint.ps1 -Lab 4 2>&1 | Tee-Object C436-work/HB-001/02-create/lab-04-test-output.txt
```

```
Set-Content C436-work/HB-001/02-create/LAB-CHECKPOINT-04.txt 'Lab 4 passed; asset pack accepted for assembly.'
```

Test it

The validator must print LAB-04 PASS. It requires the exact S01-S05, A01, and A02 set, correct types, accepted approved-rights status, files, unique IDs, and the human APPROVED_FOR_ASSEMBLY token.

Checkpoint and rejoin point

Lab checkpoint 04 is the complete 02-create stage folder. The rejoin block below restores every required accepted asset, narration, approved music brief, and human assembly decision before validating the checkpoint.

Troubleshooting

If this happens	Fix
The number of asset requests differs from the number of beats.	Check that every beat has a visual_request object and rerun from the workflow trigger without pinned output.
A generated file has no usable provenance or rights information.	Mark it rejected, restore the placeholder, and route the rights question to the owner.
Narration duration is likely too long.	Read it aloud at a natural pace, shorten the approved script, update script_version, and regenerate only the narration request.

Challenge

Add `estimated_cost_sgd` per request and a deterministic guard that compares the sum with the required `cost_budget_sgd` from the exact Lab 1 contract.

Reflection

Why is an accepted placeholder with complete provenance preferable to an impressive generated clip with unresolved rights?

Note: The complete lab and its support-file references are in `labs/lab-04-*.md`. Use only the supplied synthetic campaign data and approved accounts. Store secrets in managed credentials, keep public publishing disabled, and obtain human approval before any external release action.

Topic 03 — Editing and Assembling Videos with AI

Course coverage: Day 2 morning | 2 labs.

Automated editing | Captions, effects, and branding | Short-form assembly | AI-supported review and refinement

Key concepts

- Edit decision list — A deterministic map from source assets to timeline order, in/out points, overlays, and transitions.
- Media probe — Machine-readable evidence about codec, resolution, frame rate, duration, audio, and streams.
- Caption track — Time-aligned text stored as a portable file and optionally burned into the picture.
- Quality gate — A documented set of technical, editorial, accessibility, rights, and brand checks.

Automating Video Editing

Automated editing turns an approved manifest and edit decision list into a repeatable render. Deterministic operations such as trim, scale, crop, concatenate, overlay, caption, and audio mix are best handled by an editor or media pipeline. A model may propose the sequence or repair plan, but the render command should be explicit and logged.

Regenerating a full timeline for a small change wastes time and can introduce new errors. Manifest-driven assembly makes each input and transformation visible, supports selective reruns, and produces the same output from the same accepted files. A media probe before and after the render catches mismatched dimensions, missing audio, and duration drift.

How it works

- Validate that every required scene and audio file exists and is approved.
- Normalise aspect ratio, frame rate, codec, and naming before assembly.
- Build the timeline from a versioned edit decision list.
- Render to a draft path, then probe technical properties and duration.
- Promote only the accepted draft to the release folder.

Worked example

- The manifest lists five vertical scenes, one narration track, music, and a caption file.
- The assembly script scales to 1080x1920, applies bounded trims, mixes audio, and exports a draft.
- A duration check routes a 33.8-second result back because the contract requires no more than 32 seconds.

Decision guide

Use when	Avoid when
The repeated edit can be expressed through a stable manifest or template.	The creative decision depends on subtle performance judgment that is not encoded in the plan.
The team needs reproducible renders and traceable revisions.	The source media cannot be legally or technically processed by the chosen tool.

Practitioner quality lens

- Failure signal: A rerun changes unrelated parts of the video.
- Repair move: Pin inputs, settings, and the edit decision list to a run version.
- Quality evidence: The render log and media probe match the declared release specification.

Adding Captions, Effects, and Branding

Captions represent speech and essential audio in time-aligned text. Branding uses controlled typography, colour, logo placement, and tone. Effects should guide attention or clarify change; they are not a substitute for a coherent story. Separate caption text from styling so the same approved content can be exported as WebVTT, platform captions, or burned-in text.

Most short-form video is watched in varied sound and attention conditions. Accurate captions improve access and comprehension, while a consistent safe-area layout prevents text and controls from colliding. Automated transcription is a draft: names, numbers, timing, line breaks, and speaker meaning still require human review.

How it works

- Create captions from the approved script or a reviewed transcript.
- Check wording, timing, reading order, line length, and meaningful sound labels.
- Apply brand typography and colours within channel-safe areas.
- Use effects only when they support a story beat or viewer orientation.
- Export a portable caption file and preview the full vertical frame at phone size.

Worked example

- The WebVTT file begins with the required header and contains ordered cue timings.
- On-screen keywords complement rather than duplicate the complete caption line.
- The logo and lower-third remain clear of common interface overlays.

Decision guide

Use when	Avoid when
The transcript can be reviewed against the final audio.	The system guesses inaudible speech or decorative text hides the subject.
Brand assets and usage rules are approved and available.	A generated logo, typeface, or sound effect has uncertain rights.

Practitioner quality lens

- Failure signal: Captions are accurate but unreadable on a phone or out of sync after an edit.
- Repair move: Regenerate timings from the final audio and recheck safe area, contrast, and line breaks.
- Quality evidence: The final file passes text, timing, contrast, and mobile-preview checks.

Assembling Short-Form Videos

Assembly is the editorial act of making every visual, spoken line, caption, and sound serve one promise. The hook establishes relevance, the body delivers proof through a clear sequence, and the close completes the promise with a proportionate next action. Rhythm comes from information change, not from arbitrary rapid cuts.

An agent can detect missing files, long gaps, repeated shots, or timing mismatches, but it cannot own the final communication judgment. The creator must watch the complete video as a viewer, with sound on and off, and confirm that the story remains understandable, truthful, and appropriately paced.

How it works

- Start with the target promise and remove any beat that does not support it.
- Align narration, picture, captions, and sound by function rather than repetition.
- Use visual continuity and clear transitions to preserve orientation.
- Preview from start to finish without stopping, then record only observable issues.
- Apply the smallest revision that fixes the stated issue and rerender.

Worked example

- The opening shows the bitter-cup problem while narration names the viewer situation.
- Three proof beats demonstrate variables in the same order as the spoken explanation.
- The close summarises the checklist and invites the viewer to save it.

Decision guide

Use when	Avoid when
The script, storyboard, media, and audio are approved enough for a complete draft.	Essential evidence is missing and the edit would disguise that gap.
The team can preview on the intended aspect ratio and device.	The workflow optimises only for cut frequency or novelty.

Practitioner quality lens

- Failure signal: The edit is energetic but the viewer cannot restate the three fixes.
- Repair move: Restore causal order and remove decorative elements that compete with proof.
- Quality evidence: A cold viewer identifies the promise, proof, and next action without explanation.

Reviewing and Refining with AI

A review agent inspects the draft against a declared rubric and returns evidence, severity, location, and a bounded repair suggestion. It may compare the script to captions, probe the file, detect missing manifest entries, or flag brand and rights questions. It does not give itself permission to approve or publish its own work.

Unstructured feedback such as 'make it more engaging' causes uncontrolled rewrites. An issue register turns observations into reproducible decisions: issue ID, category, evidence, severity, owner, fix, and recheck result. Independent checks reduce the risk that the same assumptions survive from generation into review.

How it works

- Run deterministic technical and manifest checks first.
- Review story, claims, captions, rights, privacy, branding, and disclosure separately.
- Record each issue with timecode or asset reference and supporting evidence.
- Assign an owner and apply the smallest controlled change.
- Rerun affected checks and obtain human approval on the complete final preview.

Worked example

- The agent flags a caption mismatch at 00:12, a missing rights note for asset S03, and a 1.8-second duration overrun.
- The editor fixes only those items and links the new render to the same run.
- A person confirms the whole video after the automated checks return clear.

Decision guide

Use when	Avoid when
The rubric, evidence sources, and severity thresholds are explicit.	The reviewer shares the same hidden context and merely confirms its own output.
A human remains accountable for ambiguous creative and release decisions.	A score is used without evidence, location, or a repair path.

Practitioner quality lens

- Failure signal: The system gives a high quality score while required release evidence is missing.
- Repair move: Use blocking gates for mandatory fields and evidence-linked findings for judgment.
- Quality evidence: Every cleared issue has a recorded recheck and the final approval names a person.

Lab 5 — Assemble and Probe the Captioned Vertical Video

Learning outcome: LO3: assemble a reproducible short-form video with captions, branding, and technical evidence.

Goal: Render a 9:16 draft from approved checkpoint files and prove its technical properties.

You will use the supplied PowerShell and FFmpeg assembly path to verify the accepted manifest sources, create deterministic scene clips, add narrated audio and a muxed WebVTT-derived caption track. You will then probe the draft and save machine-readable technical evidence.

What you'll build

A vertical-draft-v1.mp4, captions-v1.vtt, edit-decision-list.json, ffprobe-v1.json, render-log-v1.txt, and render-evidence-v1.json in stage folder 03-edit. (Tools: PowerShell, FFmpeg, FFprobe, supplied assembly script.)

Prerequisites

- Complete Lab 4 or restore the approved placeholder asset pack.
- If FFmpeg is missing on Windows, run `winget install --id Gyan.FFmpeg -e`; on macOS, run `brew install ffmpeg`. Reopen the terminal.
- Run `ffmpeg -version` and `ffprobe -version` successfully before assembly.
- Confirm `labs/assets/assemble-harbour-bean.ps1` and the Lab 3 `captions-script-v1.vtt` are present. If installation is not possible, use the supplied `labs/assets/rejoin/lab-05` draft and probe package.

Step-by-step

1. Create the edit checkpoint and copy the controlled inputs

```
New-Item -ItemType Directory -Force -Path
C436-work/HB-001/03-edit/input/media,C436-work/HB-001/03-edit/output | Out-Null

Copy-Item -Recurse -Force C436-work/HB-001/02-create/media/* C436-work/HB-001/03-
edit/input/media/

Copy-Item C436-work/HB-001/02-create/asset-manifest.csv
C436-work/HB-001/03-edit/input/asset-manifest.csv

Copy-Item C436-work/HB-001/02-create/narration.txt
C436-work/HB-001/03-edit/input/narration.txt

Copy-Item labs/assets/narration-fallback.wav
C436-work/HB-001/03-edit/input/narration-fallback.wav

Copy-Item C436-work/HB-001/02-create/captions-script-v1.vtt C436-work/HB-001/03-
edit/captions-v1.vtt

Copy-Item labs/assets/edit-decision-list-approved.json
C436-work/HB-001/03-edit/edit-decision-list.json
```

2. Validate the edit decision list

```
Get-Content -Raw C436-work/HB-001/03-edit/edit-decision-list.json | ConvertFrom-
Json | Select-Object
run_id,version,target_duration_seconds,width,height,frame_rate | Format-List
```

3. Inspect the caption header and cues

```
Get-Content C436-work/HB-001/03-edit/captions-v1.vtt | Select-Object -First 20
```

4. Run the deterministic assembly script

```
PowerShell -ExecutionPolicy Bypass -File labs/assets/assemble-harbour-bean.ps1
-ProjectRoot C436-work/HB-001/03-edit
```

5. Confirm the expected render files exist

```
Get-Item
C436-work/HB-001/03-edit/output/vertical-draft-v1.mp4,C436-work/HB-001/03-edit/
output/render-log-v1.txt | Select-Object Name,Length,LastWriteTime
```

6. Probe the rendered video to JSON

```
ffprobe -v quiet -print_format json -show_format -show_streams
C436-work/HB-001/03-edit/output/vertical-draft-v1.mp4 | Set-Content
C436-work/HB-001/03-edit/output/ffprobe-v1.json
```

7. Bind the actual MP4, probe, captions, and log into render evidence

```
PowerShell -ExecutionPolicy Bypass -File labs/assets/write-render-evidence.ps1
-ProjectRoot C436-work/HB-001/03-edit -Version v1
```

8. Run the fail-closed Lab 5 validator and retain technical evidence

```
PowerShell -ExecutionPolicy Bypass -File labs/assets/validate-lab-checkpoint.ps1  
-Lab 5 2>&1 | Tee-Object C436-work/HB-001/03-edit/lab-05-test-output.txt
```

9. Preview the complete draft with sound on and off

Open C436-work/HB-001/03-edit/output/vertical-draft-v1.mp4 in a local media player. Confirm every scene appears in edit-decision-list order and the visual treatment does not obscure the subject.

10. Record the assembly decision

Create assembly-approval.md with asset-manifest-v1, edl-v1, script-v1, render-evidence-v1, technical/editorial findings, and Decision=READY_FOR_QUALITY_REVIEW. Then write LAB-CHECKPOINT-05.txt.

Test it

The validator must print LAB-05 PASS. It hashes and inspects the actual MP4, probe, captions, and log; asserts H.264, 1080x1920, 30 fps, yuv420p, 30-second duration, audio/subtitles, ordered VTT, and approval.

Checkpoint and rejoin point

Lab checkpoint 05 is the complete 03-edit stage folder. The block below restores the mutually hashed video, probe, caption, log, and evidence files plus the human assembly decision before running the validator.

Troubleshooting

If this happens	Fix
PowerShell cannot find ffmpeg or ffprobe.	Run the exact installer command in the prerequisites, reopen PowerShell, and rerun the version checks. If installation is unavailable, run the complete Checkpoint and Rejoin Point block; it restores all five mutually hashed evidence files and the assembly approval before validation.
The render is landscape or square.	Confirm the edit decision list width is 1080 and height is 1920, then remove the incomplete output and rerun the supplied script.
The caption cues no longer match the edited timing.	Update the cue times against the final narration and scene order, save a new caption version, and record the change before review.

Challenge

Copy the EDL to edit-decision-list-v2.json, change adjacent scene/cue boundaries while preserving 30 seconds, copy captions-v1.vtt to captions-v2.vtt and update those boundaries, then run with -Version v2 -EdlPath C436-work/HB-001/03-edit/edit-decision-list-v2.json.

Reflection

Which editing operations were safer as deterministic commands than as open-ended agent decisions?

Note: The complete lab and its support-file references are in labs/lab-05-*.md. Use only the supplied synthetic campaign data and approved accounts. Store secrets in managed credentials, keep public publishing disabled, and obtain human approval before any external release action.

Lab 6 — Run the Independent Video Review Gate and Repair One Finding

Learning outcome: LO3: review and refine a video through technical, editorial, accessibility, brand, rights, and release gates.

Goal: Produce an evidence-linked issue register, repair a controlled defect, and obtain a human release-readiness decision.

You will import an n8n review workflow that reads exact technical, caption, and manifest evidence, validates 28–32 seconds plus ordered contiguous cues, creates blocking and advisory findings, and never self-approves. You will inject a controlled defect, verify that the gate blocks, repair the data, rerun, and record the final human decision after watching the complete draft.

What you'll build

A review-agent workflow export, issue-register-v1.json, issue-register-v2.json, repair-log.md, and final-review-approval.json. (Tools: n8n, FFprobe evidence, text editor, local media player.)

Prerequisites

- Complete Lab 5 or restore Lab checkpoint 05 from stage folder 03-edit with a valid draft and probe.
- Confirm labs/assets/video-review-agent-workflow.json and review-rubric.csv are present.
- Keep the review workflow inactive and do not connect publishing tools.

Step-by-step

1. Import and rename the review workflow

In n8n, select Workflows > Create Workflow, import labs/assets/video-review-agent-workflow.json, rename it C436-HB-001-Video-Review-Gate, save it, and keep it inactive.

2. Inspect the gate structure

Confirm it consumes exact FFprobe, WebVTT, and manifest evidence; enforces 28-32 seconds plus five ordered, positive, contiguous cues; keeps approval pending; and never emits release_approved.

3. Paste exact prior evidence and create a controlled missing-rights case

Open Load Exact Review Evidence. Paste the complete Lab 5 ffprobe-v1.json, Lab 3 captions-script-v1.vtt, and Lab 4 asset-manifest.csv into the three named constants. In only the pasted manifest copy, change S01 rights_status to review_required and save.

4. Run the blocked review and save its output

Select Test Workflow. Open Emit Issue Register, copy the complete JSON output, and save it to C436-work/HB-001/03-edit/issue-register-v1.json.

5. Verify the blocking finding

```
$issues = Get-Content -Raw C436-work/HB-001/03-edit/issue-register-v1.json |  
ConvertFrom-Json
```

```
$issues | Select-Object run_id,review_status,release_allowed
```

```
$issues.findings | Format-Table  
issue_id,category,severity,evidence,required_action
```

6. Repair the controlled defect

Return to Load Exact Review Evidence and repaste the unchanged exact Lab 4 asset-manifest.csv. Record the repair in C436-work/HB-001/03-edit/repair-log.md under issue RV-RIGHTS.

7. Rerun the review and save the clear result

Select Test Workflow again. Save the Emit Issue Register JSON output as C436-work/HB-001/03-edit/issue-register-v2.json.

8. Verify that automation stops at review clear

```
$clear = Get-Content -Raw C436-work/HB-001/03-edit/issue-register-v2.json |  
ConvertFrom-Json
```

```
if ($clear.review_status -ne 'review_clear' -or $clear.release_allowed -ne $false  
-or $clear.owner_approval_status -ne 'pending') { throw 'Automated review did not  
stop before human approval.' }
```

9. Conduct the human full-length review

Watch vertical-draft-v1.mp4 once with sound and once muted. Review the supplied rubric for story, claims, captions, mobile readability, brand, privacy, rights, disclosure, and technical output. Record timecoded observations.

10. Record the scoped decision

```
$path='C436-work/HB-001/03-edit/final-review-approval.json'
```

```
Copy-Item labs/assets/final-review-approval-template.json $path
```

```
$a=Get-Content -Raw $path | ConvertFrom-Json
```

```
$a.reviewer='<YOUR_NAME>'; $a.reviewed_at=[datetimeoffset]::UtcNow.ToString('o');  
$a.decision='APPROVED_FOR_PRIVATE_RELEASE_PACKAGE'; $a.scope='private release  
package only; no public posting'; $a.reviewed_video_sha256=(Get-FileHash  
-Algorithm SHA256 C436-work/HB-001/03-edit/output/vertical-draft-  
v1.mp4).Hash.ToLowerInvariant(); $a.unresolved_blockers=@()
```

```
$a | ConvertTo-Json -Depth 10 | Set-Content $path
```

11. Export the reviewed workflow

Use n8n Download and save C436-work/HB-001/03-edit/video-review-agent-reviewed.json. Then run labs/assets/validate-lab-checkpoint.ps1 -Lab 6 and retain lab-06-test-output.txt.

Test it

The validator must print LAB-06 PASS: rights, invalid duration, or bad caption timing must block; repaired evidence stays non-releasing; the human JSON approval must bind the actual video hash and private-only scope.

Checkpoint and rejoin point

Lab checkpoint 06 adds both issue registers, the repair log, structured human final review, and exported workflow to C436-work/HB-001/03-edit. Keep v1 evidence to prove the defect. The trainer-approved rejoin block below restores exact review artifacts and verifies that the final approval names the SHA-256 of the supplied Lab 5 MP4.

Troubleshooting

If this happens	Fix
The clear run still contains the rights blocker.	Confirm the edited evidence node was saved and run from the workflow trigger rather than a downstream node with pinned data.
The review result sets release_allowed=true.	Stop. Restore the supplied workflow and confirm the final deterministic gate always sets release_allowed=false pending a separate approval token.
The video preview reveals a new issue not in the automated output.	Record it with evidence and severity, repair it, and rerun only the affected technical checks plus the complete human preview.

Challenge

Add a blocking rule for duplicate WebVTT cue identifiers, prove it with a controlled duplicate cue ID, then restore the approved caption file and confirm the full review returns review_clear.

Reflection

What important quality judgment remained invisible to the automated review evidence?

Note: The complete lab and its support-file references are in labs/lab-06-*.md. Use only the supplied synthetic campaign data and approved accounts. Store secrets in managed credentials, keep public publishing disabled, and obtain human approval before any external release action.

Topic 04 — Automating and Scaling Video Production

Course coverage: Day 2 afternoon | 2 labs.

Multi-step video agents | Publishing and scheduling | Performance analysis | Scalable content pipelines

Key concepts

- Orchestrator — The workflow component that routes state between specialist stages and enforces gates.
- Approval token — A recorded, scoped decision that authorises one release action for one approved package.
- Metric contract — A definition of the decision, grain, window, numerator, denominator, and exclusions for each KPI.
- Scaling guardrail — A limit on volume, spend, retries, permissions, or variance that grows with automation.

Building Multi-Step Video Agents

A multi-step video system may use a coordinator plus specialist research, script, asset, edit, review, and release stages. Specialisation is valuable when each role has a distinct tool set and output contract. The coordinator should route state and enforce policies, not rewrite every artifact.

Adding agents increases hand-offs, cost, latency, and opportunities for inconsistent assumptions. Start with one workflow and split a stage only when the separation improves control, parallel work, specialist tooling, or evaluation. Shared run state must distinguish approved artifacts from drafts.

How it works

- Define a state machine with permitted transitions and blocking conditions.
- Give each specialist the minimum context and tools required for its stage.
- Validate every hand-off against the shared schema and artifact version.
- Cap iterations and route unresolved work to a rework queue or human owner.
- Trace tool calls, costs, decisions, errors, and approvals under one run ID.

Worked example

- HB-001 moves `brief_approved` -> `script_ready` -> `assets_ready` -> `draft_ready` -> `release_ready`.
- A rights flag prevents the release stage from running even when all media files exist.
- The coordinator sends the issue back to the asset owner rather than regenerating the script.

Decision guide

Use when	Avoid when
Specialist roles have distinct contracts, tools, or review criteria.	One prompt and one tool would solve the bounded task more reliably.
The orchestrator can persist state and enforce permitted transitions.	Agents share unrestricted credentials or can silently overwrite accepted artifacts.

Practitioner quality lens

- Failure signal: Agents loop or disagree about which artifact is current.
- Repair move: Use a state machine, immutable versions, and one authoritative manifest.
- Quality evidence: The complete run can be reconstructed from state transitions and logs.

Publishing and Scheduling Across Platforms

Publishing is an external, high-impact action. The release stage should receive only an approved video, captions, metadata, disclosure decision, rights record, target account, privacy setting, and schedule. The default learning path produces a dry-run request and keeps visibility private until an authorised person reviews platform-specific fields and consents.

Platforms have different permissions, quotas, disclosure fields, and audit requirements. Automating the final click without checking creator information, audience settings, privacy, and synthetic-media disclosure can cause irreversible mistakes. A scoped approval token and idempotency key ensure one approved package produces at most one intended post.

How it works

- Validate the release package and platform-specific required fields.
- Query the authorised account or creator settings where the API requires it.
- Show the exact title, description, captions, disclosure, privacy, and schedule to a person.
- Record approval, then initialise one private or scheduled upload with an idempotency key.
- Poll processing status, record the returned post ID, and route errors without duplicate posts.

Worked example

- The C436 lab creates a private dry-run package for YouTube and a TikTok request preview.

- The release remains blocked until disclosure, rights, and owner_approval fields are complete.
- The workflow records the intended target and package checksum before any live integration is enabled.

Decision guide

Use when	Avoid when
The account owner has authorised the app and can preview every required field.	The workflow would post publicly without explicit consent and a complete preview.
Private or draft mode, error handling, and duplicate prevention are available.	Credentials, disclosure, rights, or platform audit requirements are unresolved.

Practitioner quality lens

- Failure signal: A retry produces two uploads or exposes an unreviewed caption.
- Repair move: Gate the action with approval plus an idempotency key and reconcile the returned post ID.
- Quality evidence: The release log links one approved package to one intended platform action.

Analysing Performance

Performance analysis starts with a decision and a metric contract. Reach, starts, watch time, average view duration, completion, saves, shares, comments, and downstream actions describe different parts of audience response. The agent should calculate defined metrics at a consistent grain, compare appropriate windows, and separate observation from explanation.

A large view count does not prove that a creative choice caused success. Platform metric definitions and counting rules can change, and small samples are unstable. A useful analysis agent preserves denominators, dates, segment, content version, and data source, then proposes a limited next test instead of declaring a universal rule.

How it works

- State the decision and choose one primary metric plus guardrails.
- Validate dates, video IDs, denominators, missing values, and metric definitions.
- Compare like with like and calculate rates at the intended grain.
- Describe observed differences before proposing possible drivers.
- Recommend one controlled creative or workflow change with a measurement window.

Worked example

- The Harbour Bean synthetic dataset compares eight posts by hook family and duration.
- The agent calculates completion and save rates from the supplied counts, then flags low-impression rows.
- It recommends testing the strongest clear-promise hook while holding topic and duration band stable.

Decision guide

Use when	Avoid when
The data source, metric definitions, window, and comparison grain are known.	Denominators are missing or metrics from different platforms are treated as identical.
A proposed action can be tested and reviewed against guardrails.	A single high-performing post is being treated as causal proof.

Practitioner quality lens

- Failure signal: The dashboard ranks videos by a rate calculated from incompatible denominators.
- Repair move: Write the metric contract and validate each row before analysis.
- Quality evidence: Every recommendation cites a defined metric, segment, window, caveat, and next test.

Scaling Your Video Content Pipeline

Scaling means increasing useful throughput without losing evidence, control, or quality. The team standardises production contracts, templates, reusable agent tools, asset naming, review rubrics, and telemetry. Work is queued and prioritised; capacity and cost budgets are visible; exceptions are handled explicitly.

Volume magnifies small defects. A weak prompt creates many weak scripts, a permissive credential multiplies risk, and a missing rights record blocks an entire catalogue. Scale should follow demonstrated reliability at lower volume. Versioned templates, sampling, canary releases, and stop conditions keep growth reversible.

How it works

- Measure baseline lead time, rework, cost, quality findings, and release errors.
- Standardise only stages with stable inputs, outputs, and owner acceptance.
- Introduce queues, concurrency limits, budgets, and priority rules.
- Use canary batches and sample-based human review before increasing volume.
- Monitor drift, rights expiry, tool changes, and incidents; pause when thresholds fail.

Worked example

- The team moves from one video to a three-video weekly batch using the same approved production contract.
- Generation is capped per run, every fifth draft receives an additional cold review, and public release stays human-approved.
- A scorecard tracks cycle time, cost per accepted video, rework rate, blocked rights items, and post-release learning.

Decision guide

Use when	Avoid when
The pilot is reliable, measurable, and recoverable.	Quality evidence is incomplete or rework already consumes more time than the workflow saves.
Owners, limits, incident response, and manual fallback are in place.	The plan increases permissions or public actions faster than monitoring and review capacity.

Practitioner quality lens

- Failure signal: Output volume rises while accepted-video cost and rework also rise.
- Repair move: Throttle the queue and fix the earliest stage producing repeated defects.
- Quality evidence: Throughput improves while guardrail metrics remain within agreed limits.

Lab 7 — Orchestrate the Private Release Package with Human Approval

Learning outcome: LO4: orchestrate multi-step agents and prepare safe platform-specific publishing actions.

Goal: Build a release-ready package that is blocked until a person approves one scoped private action.

You will import a release orchestrator, inspect its state transitions, validate video, caption, metadata, rights, and disclosure fields, then exercise denial and approval paths. The lab creates a dry-run package only; no public account or live posting credential is used.

What you'll build

A release-package.json, denied-approval.json, private-release-approval.json, and release-orchestrator-reviewed.json in stage folder 04-release. (Tools: n8n, supplied release workflow, reviewed video package.)

Prerequisites

- Complete Lab 6 or restore Lab checkpoint 06 with final-review-approval.json bound to the reviewed video.
- Confirm labs/assets/release-orchestrator-workflow.json and release-metadata-template.json are present.
- Do not connect a YouTube, TikTok, or other publishing credential during the required lab path.

Step-by-step

1. Create the release checkpoint

```
New-Item -ItemType Directory -Force C436-work/HB-001/04-release | Out-Null
```

```
Copy-Item labs/assets/release-metadata-template.json  
C436-work/HB-001/04-release/release-metadata.json
```

2. Complete the release metadata without account secrets

Open release-metadata.json and set title, description, caption_path='02-create/captions-script-v1.vtt', video_path='03-edit/output/vertical-draft-v1.mp4', rights_evidence_path='02-create/asset-manifest.csv', final_review_approval_path='03-edit/final-review-approval.json', and package_owner. Retain typed private/null/true values and public_release_allowed=false.

3. Build and hash the canonical release package manifest

```
PowerShell -ExecutionPolicy Bypass -File labs/assets/build-release-package.ps1  
-WorkRoot C436-work/HB-001
```

```
Get-Content -Raw C436-work/HB-001/04-release/release-package-manifest.json |  
ConvertFrom-Json | Select-Object  
run_id,package_version,title,privacy_status,contains_synthetic_media
```

4. Import and inspect the release orchestrator

In n8n, create a workflow from labs/assets/release-orchestrator-workflow.json, rename it C436-HB-001-Private-Release-Orchestrator, save it, and keep it inactive. Confirm the states are release_input -> validation -> awaiting_human_approval -> dry_run_ready or denied.

5. Run the input validation path

Open Load Exact Release Metadata and paste the complete release-metadata.json between the releaseText backticks. Select Test Workflow and inspect Emit Release Package.

6. Save the awaiting-approval package

Copy the Emit Release Package JSON output to C436-work/HB-001/04-release/release-package.json. Confirm publish_allowed=false and status=awaiting_human_approval.

7. Exercise the denial path

Edit release-metadata.json to set human_decision='deny' and decision_reason='Controlled lab denial - verify no action'. Repaste the complete file, run, and save the complete output as C436-work/HB-001/04-release/denied-approval.json.

8. Verify denial causes no external action

```
$denied = Get-Content -Raw C436-work/HB-001/04-release/denied-approval.json |  
ConvertFrom-Json
```

```
$denied | Select-Object status,publish_allowed,external_action_count
```

```
if ($denied.external_action_count -ne 0) { throw 'Denial must cause zero external  
actions' }
```

9. Create a scoped private approval

```
$path='C436-work/HB-001/04-release/release-metadata.json'
```

```
$m=Get-Content -Raw $path | ConvertFrom-Json
```

```
$m.human_decision='approve_private_dry_run'; $m.decision_reason='Reviewed exact  
package for private dry run only'; $m.approval_scope='HB-001; private dry-run  
request preview; one package; no external execution';  
$m.approval_package_sha256=$m.package_sha256;  
$m.approval_expires_at=[datetimeoffset]::UtcNow.AddHours(8).ToString('o')
```

```
$m | ConvertTo-Json -Depth 10 | Set-Content $path
```

Repaste the complete file into Load Exact Release Metadata and run.

10. Save and verify the approved dry-run output

Save the final output as C436-work/HB-001/04-release/private-release-approval.json. Confirm dry_run_ready, non-publishing, zero actions, a non-empty idempotency key, and disclosure-complete non-executing platform previews.

11. Export the reviewed orchestrator

Use n8n Download and save C436-work/HB-001/04-release/release-orchestrator-reviewed.json.

12. Run the fail-closed Lab 7 validator and retain the evidence

```
PowerShell -ExecutionPolicy Bypass -File labs/assets/validate-lab-checkpoint.ps1  
-Lab 7 2>&1 | Tee-Object C436-work/HB-001/04-release/lab-07-test-output.txt
```

```
Set-Content C436-work/HB-001/04-release/LAB-CHECKPOINT-07.txt 'Lab 7 passed; hash-  
bound private dry-run package retained.'
```

Test it

The validator must print LAB-07 PASS. It recomputes the canonical package and every video/caption/rights/final-review hash, requires an exact approval hash plus future expiry, and validates zero-action platform previews.

Checkpoint and rejoin point

Lab checkpoint 07 is stored in stage folder C436-work/HB-001/04-release. It contains both decision paths and the private dry-run package. A live integration may be added later only by an authorised owner using current official platform documentation and managed credentials.

Troubleshooting

If this happens	Fix
The validator reports a missing disclosure decision.	Set <code>contains_synthetic_media</code> explicitly to true or false based on the reviewed content; never leave it implicit.
The approved dry run sets <code>publish_allowed=true</code> .	Stop and restore the supplied workflow. This lab never authorises a live public action.
The idempotency key changes every time the same package is retried.	Build it from the stable run ID, target, and package checksum rather than the current timestamp.

Challenge

Set `approval_expires_at` to one minute in the past and prove the package is blocked with `approval_hash_or_expiry_invalid` and zero actions, then restore a future expiry.

Reflection

Which release fields must a person see together before an approval can be considered informed?

Note: The complete lab and its support-file references are in `labs/lab-07-*.md`. Use only the supplied synthetic campaign data and approved accounts. Store secrets in managed credentials, keep public publishing disabled, and obtain human approval before any external release action.

Lab 8 — Analyse Synthetic Performance and Build the Scaling Control Plan

Learning outcome: LO4: analyse performance with metric contracts and scale the pipeline through measurable guardrails.

Goal: Turn synthetic post data into a bounded next test and a scaling decision supported by operational evidence.

You will import a deterministic analytics workflow, validate metric definitions and denominators, calculate completion and save rates, inspect low-volume caveats, and produce one creative experiment. You will then complete a scaling scorecard covering throughput, cost, rework, quality, rights, incidents, and rollback.

What you'll build

A metric-contract.json, analytics-result.json, next-test.md, scaling-scorecard.csv, and integrated-handover.md in stage folder 05-learn. (Tools: n8n, supplied synthetic analytics CSV, spreadsheet or text editor.)

Prerequisites

- Complete Lab 7 or restore Lab checkpoint 07.
- Confirm labs/assets/synthetic-video-analytics.csv, analytics-scale-workflow.json, and scaling-scorecard-template.csv are present.
- Treat all analytics as synthetic; do not infer facts about real people or accounts.

Step-by-step

1. Create the learning checkpoint and copy the supplied data

```
New-Item -ItemType Directory -Force C436-work/HB-001/05-learn | Out-Null
```

```
Copy-Item labs/assets/synthetic-video-analytics.csv  
C436-work/HB-001/05-learn/synthetic-video-analytics.csv
```

```
Copy-Item labs/assets/synthetic-pipeline-operations.csv C436-work/HB-001/05-learn/synthetic-pipeline-operations.csv
```

```
Copy-Item labs/assets/metric-contract-approved.json  
C436-work/HB-001/05-learn/metric-contract.json
```

```
Copy-Item labs/assets/scaling-scorecard-template.csv  
C436-work/HB-001/05-learn/scaling-scorecard.csv
```

```
Copy-Item labs/assets/scale-decision-template.json  
C436-work/HB-001/05-learn/scale-decision.json
```

2. Inspect data grain and required denominators

```
Import-Csv C436-work/HB-001/05-learn/synthetic-video-analytics.csv | Format-Table  
video_id,hook_family,duration_seconds,views,completed_views,saves,shares
```

3. Review the metric contract before calculation

```
Get-Content -Raw C436-work/HB-001/05-learn/metric-contract.json | ConvertFrom-Json  
| Select-Object decision,primary_metric,minimum_views,comparison_grain>window |  
Format-List
```

4. Import the analytics workflow

In n8n, create a workflow from labs/assets/analytics-scale-workflow.json, rename it C436-HB-001-Analytics-and-Scale, save it, and keep it inactive.

5. Inspect deterministic calculations

Confirm `completion_rate=completed_views/views` and `save_rate=saves/views` after excluding rows below `minimum_views`. Confirm `cost_per_accepted_video=sum(cost_sgd)/accepted_count` and any rights, review, duplicate-release, rollback, or capacity blocker forces HOLD.

6. Paste the exact source artifacts into the workflow

Open Load Exact Analytics Operations and Contract. Paste the complete synthetic-video-analytics.csv, synthetic-pipeline-operations.csv, and metric-contract.json into their named constants. Do not retype or use a subset.

7. Run the analytics workflow and save the result

Select Test Workflow. Open Emit Analysis and save its complete JSON output as C436-work/HB-001/05-learn/analytics-result.json.

8. Verify rates and caveats

```
$result = Get-Content -Raw C436-work/HB-001/05-learn/analytics-result.json |  
ConvertFrom-Json  
  
$result.summary_by_hook | Format-Table  
hook_family,video_count,total_views,completion_rate,save_rate  
  
$result.caveats  
  
if (-not $result.low_volume_video_ids) { Write-Host 'No low-volume rows in this  
supplied dataset' }
```

9. Calculate baseline, pilot, cost, rework, blocker, rollback, and capacity evidence

```
$ops=@(Import-Csv C436-work/HB-001/05-learn/synthetic-pipeline-operations.csv);  
$base=@($ops|Where-Object phase -eq 'baseline'); $pilot=@($ops|Where-Object phase  
-eq 'pilot'); $accepted=@($ops|Where-Object accepted_status -eq 'accepted');  
$rework=@($ops|Where-Object rework_required -eq 'true'); $blockers=@($ops|Where-  
Object {[int]$_unresolved_rights_items -gt 0 -or [int]$_blocking_review_findings  
-gt 0});  
$summary=[ordered]@{baseline_job_count=$base.Count;pilot_job_count=$pilot.Count;ba  
seline_average_cycle_minutes=[math]::Round(((($base|Measure-Object cycle_minutes  
-Average).Average),2);pilot_average_cycle_minutes=[math]::Round(((($pilot|Measure-  
Object cycle_minutes  
-Average).Average),2);cost_per_accepted_video_sgd=[math]::Round(((($ops|Measure-  
Object cost_sgd
```

```
-Sum).Sum/$accepted.Count),2);rework_rate=[math]::Round(($rework.Count/
$ops.Count),4);blocking_job_ids=@($blockers.job_id);duplicate_release_actions=[int
](($ops|Measure-Object duplicate_release_actions -Sum).Sum);rollback_all_tested=(-
not ($ops|Where-Object rollback_tested -ne
'true'));human_review_capacity_sufficient=(-not ($ops|Where-Object {[int]
$.human_review_capacity_slots -lt [int]$.human_reviews_required})); $summary|
ConvertTo-Json -Depth 8|Set-Content C436-work/HB-001/05-learn/operational-
summary.json

$summary
```

10. Write one bounded next test

Create next-test.md with Decision, Observation, Caveat, Hypothesis, Single change, Held constant, Primary metric, Guardrails, Minimum sample, Review date, and Stop rule. Change only hook family; hold topic and duration band constant.

11. Complete the scaling scorecard

Use operational-summary.json and the phase/capacity columns to complete all eight rows. Set baseline, pilot_evidence, threshold, owner, status, and action with no placeholders. At least the rights/review rows must be HOLD while the supplied pilot blocker remains.

12. Make the scale decision

Open scale-decision.json. Set scale_decision to HOLD for the supplied blocker, add reason and next_owner, and retain canary_limit=3, visibility=private, human_review_rate=1.0, and rollback path. Only a later clean pilot may use PILOT_3_PER_WEEK or SCALE_WITH_LIMITS.

13. Create the integrated handover

Write integrated-handover.md with literal LAB-CHECKPOINT-01 through LAB-CHECKPOINT-08 paths plus headings: artifact versions, unresolved issues, enabled tools, disabled tools, approval scope, metric decision, scale decision, rollback path, and next owner.

14. Export the reviewed analytics workflow

Use n8n Download and save C436-work/HB-001/05-learn/analytics-scale-reviewed.json. Run labs/assets/validate-lab-checkpoint.ps1 -Lab 8 and retain lab-08-test-output.txt.

Test it

The validator must print LAB-08 PASS. It proves low-volume exclusion, baseline/pilot, duplicate-release, rollback, and capacity gates, exact scorecard controls, the complete next test/handover, and a structured HOLD/canary decision.

Checkpoint and rejoin point

Lab checkpoint 08 is the full stage folder C436-work/HB-001/05-learn plus the integrated handover. A rejoining learner may use the approved metric contract and supplied synthetic data, but must still make and explain their own bounded next-test and scaling decisions.

Troubleshooting

If this happens	Fix
A calculated rate is greater than 1 or below 0.	Check numeric conversion and confirm the numerator cannot exceed the declared denominator; flag the row instead of repairing it silently.
The workflow recommends a winner from a low-volume row.	Apply the minimum_views rule before ranking and retain the caveat in the result.
The scale scorecard is green while rights or release evidence is missing.	Set the affected control to blocked and choose HOLD until the owner resolves and rechecks it.

Challenge

Use the supplied cost and accepted-status fields to calculate cost per accepted video, then add a canary rule limiting the first pilot to three private drafts with 100% human review.

Reflection

Which scaling metric would reveal that the workflow is producing more output but less useful accepted work?

Note: The complete lab and its support-file references are in labs/lab-08-*.md. Use only the supplied synthetic campaign data and approved accounts. Store secrets in managed credentials, keep public publishing disabled, and obtain human approval before any external release action.

Integrated Workflow Wrap-Up

The completed C436 project is not merely a generated video. It is a controlled production system whose artifacts, decisions, and evidence can be reviewed, resumed, and improved.

Minimum handover package

- Approved production contract, autonomy matrix, and workflow map.
- Source register, script, storyboard, asset manifest, and provenance notes.
- Edit decision list, caption file, technical probe, draft, and issue register.
- Release package, approval record, metric contract, analysis, and scaling scorecard.

Operational rule

When a stage cannot prove that its required input is approved, it must stop or escalate. It must not silently invent the missing fact, permission, file, or decision.

Next Steps

- Rerun the eight labs from the saved checkpoints and explain every state transition.
- Replace one mock tool with an approved live integration while preserving the same contract, limits, and evidence.
- Add three representative test jobs, including one missing-input case and one tool-failure case.
- Pilot with private outputs, review rework and cost evidence, and scale only after the guardrails remain stable.
- Review official tool and platform documentation before adapting any live API or interface shown in this guide.
- Recheck YouTube upload/privacy requirements at https://developers.google.com/youtube/v3/guides/uploading_a_video and TikTok Direct Post requirements at <https://developers.tiktok.com/doc/content-posting-api-reference-direct-post> before any authorised implementation.

Glossary

- **Agent** — A model-led system that pursues a bounded goal through instructions, tools, state, and a control loop.
- **Agent loop** — The repeated observe, plan, act, inspect, decide, and record cycle.
- **Approval token** — A recorded decision authorising one scoped action for one named artifact or package.
- **Asset manifest** — The authoritative inventory of media files, prompts, versions, provenance, rights notes, and status.

- **Autonomy matrix** — A table classifying tasks as deterministic, model-assisted, human-approved, or prohibited.
- **Checkpoint** — A saved accepted state from which the workflow can safely resume.
- **Continuity bible** — Stable visual and audio constraints reused across generated scenes.
- **Edit decision list** — A structured description of timeline order, trims, overlays, transitions, and audio.
- **Guardrail** — A rule or check that constrains inputs, tool calls, outputs, permissions, or actions.
- **Human-in-the-loop** — A workflow point where a person reviews evidence and approves, edits, rejects, or stops an action.
- **Idempotency key** — A stable identifier used to prevent a retry from creating a duplicate external action.
- **Metric contract** — A precise definition of a measure, including grain, period, numerator, denominator, and exclusions.
- **Orchestrator** — The component that routes work between stages and enforces state transitions and gates.
- **Production contract** — A structured brief containing purpose, evidence, constraints, deliverables, tools, limits, and finish conditions.
- **Provenance** — Recorded information about where an artifact came from and how it was created or changed.
- **Run ID** — A unique identifier linking the events and artifacts of one workflow execution.
- **Schema** — A definition of required fields, data types, allowed values, and relationships.
- **Tool boundary** — The documented inputs, outputs, permissions, limits, and failure behavior of a capability.
- **WebVTT** — A UTF-8 time-aligned text format commonly used for web video captions and subtitles.